THE “HOW LONG?”
DATA COLLECTION AND PREPROCESSING

The Label or target our Model will be predicting is:
→**Waiting time (in hours [for users] and minutes[for modelling and evaluation])**
Derived from :
→**Waiting Time = Exit Time - Arrival Time**

Data will be collected from from the following sources:

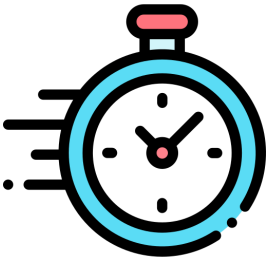
1. User reported data:

- **Users will report how long they waited after visiting a branch.**
- **Arrival timestamp**
- **Exit timestamp**

The above mission is practical, because:

- It’s low-cost, as no paid datasets are required
- It produces direct labels, the waiting time
- It scales with adoption, which is very good for the improvement of the model
- The collection of the data from the user matches the POPIA-style minimality: which states that you only collect what you need.

2. Branch information:
   → **Branch Location**
   → **Type of service:**
   - **ID application or collection**
   - **Passport application or collection**

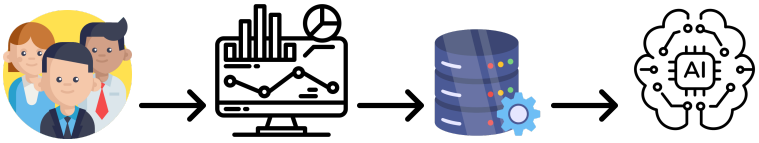


3. Time-based data:
   - **Time of day**
   - **Day of the week**
   - **Month**

4. Booking Status:
   - **Real Time information of whether the user had a booking or was a walk-in.**

Below is an illustration of our approach using a small pilot dataset to demonstrate how the model would be trained.

Visit_ID	Branch	Time	Day	Service	Booking	Crowd_level	arrival_ts	exit_ts	Weather_rain_
1	Soweto	08:00	Monday	ID	Yes	high	2026-03-24 07:4	2026-03-24 13:4	0
2	Dobsonville	14:00	Thursday	Passport	No	low	2026-03-25 07:4	2026-03-25 13:4	0.1
4	Braamfontein	10:00	Thursday	Collection	Yes	medium	2026-03-26 07:4	2026-03-26 13:4	1.45
5	Boksburg	11:00	Tuesday	ID	No	low	2026-03-27 07:4	2026-03-27 13:4	1.34
6	Ellis Park	12:00	Monday	ID	Yes	high	2026-03-28 07:4	2026-03-28 13:4	1.87
7	Pretoria	09:00	Wednesday	Collection	Yes	high	2026-03-29 07:4	2026-03-29 13:4	1.2
8	Sandton	15:00	Friday	Collection	Yes	medium	2026-03-30 07:4	2026-03-30 13:4	0
9	Randburg	13:00	Wednesday	Passport	No	high	2026-03-31 07:4	2026-03-31 13:4	1.2
10	Diepkloof	11:00	Friday	ID	No	medium	2026-04-01 07:4	2026-04-01 13:4	0



This approach is feasible because:

- Data can be collected through the app at low cost
- Users provide the actual waiting time (no manual labeling needed)
- The dataset improves as more users use the system

ADDITIONAL FEATURE FOR MORE REALISTIC PREDICTIONS

- **DHA reports:**
  → Show causes of long queues:
  - **Staffing issues**
  - **System downtime**
  - **Overcrowding**

Weather & Calendar

- **Public holidays → demand spikes**
- **Weather (rain/temp) → affects behaviour**

Data Preprocessing

Raw data collected from users may be:

- **Messy**
- **Inconsistent**

- **Sometimes incorrect**

We will preprocess the data as follows:

Step 1: Validation

- Ensure:
  - **arrival ≤ exit**
  - **valid branch/service types**
- Reject bad data

Tools:

- Python pandas library
- Python numpy library

Step 2: Standardisation

Data is converted into consistent format:

- **Branch names** → `branch_id`
- **Services** → **fixed categories**

Tools:

- pandas

Step 3: Missing Values

- **Keep nulls for optional fields**
- **Add flags:**
  - `counters_open_missing = 1`

Step 4: Outlier Handling

- **Remove impossible values:**
  - **wait > 10 hours**
  - **Wait <= 0 hours**
- **Cap extreme values**
- **Remove duplicates**

Step 5: Aggregation

Group by:

- branch
- hour
- day

Then compute:

- average wait (last 28 days)
- number of reports
- % walk-ins

Step 6: Time Features

- hour of day
- day of week
- month-end flag
- holiday flags

Step 7: Encoding

Tools:

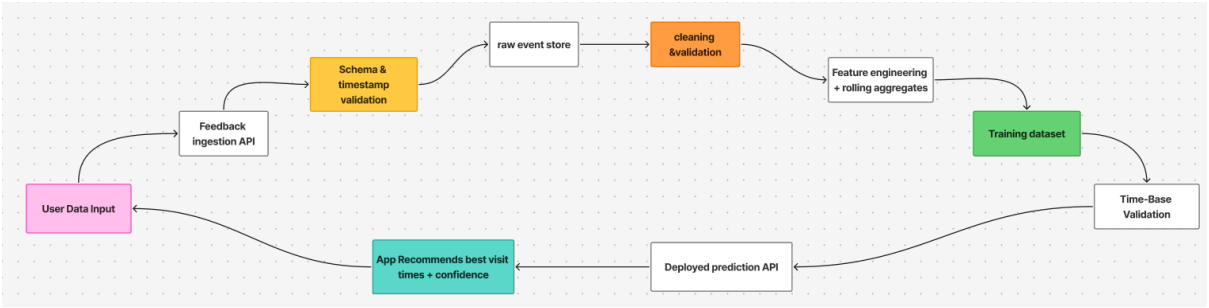
- sklearn.preprocessing

- Convert categories to numbers

Step 8: Smoothing

- **Use moving averages (EMA) method**
- **Only predict if enough data exists**

Data Pipeline Flow:



DATA PIPELINE TOOLS:

Data Processing:
→ Pandas, NumPy
Feature Engineering:
→ Pandas, datetime
Modeling:
→ Scikit-learn, XGBoost / LightGBM
API / Backend:
→ Flask / FastAPI

Storage:

→ PostgreSQL / Firebase